REFERÊNCIA TÉCNICA PERMANENTE

Catálogo de Skills & Engenharia

Dashboard Velocidades CSG • Conjunto de Ferramentas, Agentes e Metodologias

PROJETO	SKILLS LOCAIS	DIRETÓRIO	DATA DE GERAÇÃO
Dashboard CSG	**41 Especializadas**	**.agents/skills/**	**Outubro / 2026**

1. Design & Desenvolvimento Frontend

`uiux-designer`

Design System & UI

Guia e catálogo completo de UI/UX com mais de 50 estilos visuais, 97 paletas harmoniosas e 57 combinações tipográficas. Excelente para estruturar interfaces profissionais em React, Next.js, Dashboards e aplicações analíticas.

Quando usar: Ao criar ou reestilizar telas, escolher paletas de cores, definir fontes e garantir consistência visual no Dashboard.

`design-taste-frontend`

Anti-Slop Art Direction

Direção de arte premium (tasteskill v2) baseada no estudo de 247 sites premiados pelo Awwwards e One Page Love. Elimina o visual genérico e padronizado gerado por IAs, inspecionando a página em navegador real e refinando a estética iterativamente.

Quando usar: Quando um layout parecer genérico, amador ou com 'cara de template de IA', ou ao desenhar landing pages e telas de alto impacto.

`frontend-expert`

React / TypeScript

Especialista em React e TypeScript moderno. Implementa padrões de alta performance como React Suspense, useSuspenseQuery, lazy loading, TanStack Router, estilização com MUI v7 e otimização de renderização.

Quando usar: Ao programar componentes complexos, gerenciar dados assíncronos ou resolver gargalos de performance no frontend.

`prototype`

Prototipagem Rápida

Constrói protótipos descartáveis e funcionais em poucos minutos para responder perguntas de arquitetura, fluxo de usuário ou viabilidade de interface sem comprometer a base de código principal.

Quando usar: Para testar rapidamente uma ideia de tela, validar um modelo de estado ou experimentar uma nova biblioteca.

2. Planejamento, Entrevistas & Especificação

`grill-me`

Validação de Plano

Entrevista incisiva e sem rodeios para estressar e amadurecer um plano, design técnico ou ideia antes de qualquer linha de código ser escrita. Identifica furos conceituais e cenários de borda.

Quando usar: Antes de iniciar uma grande refatoração ou funcionalidade, para garantir que todas as premissas estão corretas.

`grill-with-docs`

Planejamento com ADRs

Conduz o interrogatório desafiador do grill-me enquanto documenta simultaneamente as decisões tomadas em registros de decisão de arquitetura (ADRs) e no glossário de termos.

Quando usar: Para planejar sistemas críticos onde o registro formal das decisões arquiteturais e do domínio precisa ficar documentado.

`grilling`

Teste de Estresse

Desafia agressivamente o usuário ou plano proposto contra premissas falhas, trade-offs não avaliados e complexidade acidental.

Quando usar: Sempre que você quiser que suas ideias sejam colocadas à prova sob a ótica mais crítica possível.

`loop-me`

Design de Workflows

Processo de entrevista voltado para mapear detalhadamente os fluxos de trabalho (workflows) e interações que precisam ser construídos no repositório.

Quando usar: Para desenhar processos de ponta a ponta e garantir clareza nas etapas operacionais antes da codificação.

`to-spec`

Geração de Spec

Transforma o conteúdo de uma sessão ou conversa em uma especificação técnica formal completa, pronta para publicação no gerenciador de tarefas do projeto.

Quando usar: Após discutir uma funcionalidade e precisar convertê-la em uma especificação detalhada de produto/engenharia.

`to-tickets`

Decomposição em Tarefas

Quebra uma especificação ou plano complexo em pequenos tickets operacionais (*tracer bullets*), explicitando dependências bloqueantes entre cada um.

Quando usar: Para transformar uma especificação técnica em um backlog acionável e ordenado para implementação.

`to-questionnaire`

Resolução de Incertezas

Converte decisões pendentes ou perguntas em aberto que você não consegue responder no momento em um questionário estruturado para outros participantes da equipe.

Quando usar: Ao encontrar bloqueios de negócio ou decisões que dependem de outros desenvolvedores ou gestores.

`wayfinder`

Mapeamento de Épicos

Mapeia projetos gigantescos que excedem uma única sessão de agente em um grafo de decisões e tickets rastreáveis, destravando um por um até o destino final.

Quando usar: Para grandes empreitadas de engenharia, migrações completas ou módulos inteiros.

🏛️ 3. Arquitetura & Modelagem de Domínio

`codebase-design`

Deep Modules

Princípios e vocabulário formal de design de software para criar 'módulos profundos' (*deep modules*): interfaces públicas enxutas com implementações robustas e bem encapsuladas.

Quando usar: Ao desenhar interfaces públicas de módulos, definir fronteiras de pacotes e tornar o código altamente testável.

`domain-modeling`

Linguagem Ubíqua

Constrói e preserva a precisão do modelo de domínio no projeto, mantendo os arquivos GLOSSARY.md e ADRs perfeitamente alinhados à terminologia do negócio.

Quando usar: Ao introduzir novos conceitos de domínio ou evitar discrepâncias de nomenclatura entre o código e as regras de negócio.

`improve-codebase-architecture`

Auditoria de Arquitetura

Analisa minuciosamente a base de código em busca de oportunidades de encapsulamento, exibe um relatório visual interativo em HTML e permite aprofundar nas prioridades escolhidas.

Quando usar: Para fazer um raio-x arquitetural da aplicação e planejar melhorias estruturais de médio prazo.

`setup-ts-deep-modules`

Governança de Imports

Configura o dependency-cruiser no repositório TypeScript para assegurar que cada módulo oculte seus detalhes internos e seja acessado exclusivamente por seus arquivos de entrada (index).

Quando usar: Para impedir acoplamentos indevidos e importações circulares em projetos TypeScript.

4. Implementação, Testes & Depuração

`implement`

Execução de Tarefas

Executa a codificação passo a passo baseando-se em uma especificação aprovada ou em um conjunto de tickets, garantindo aderência rigorosa ao escopo delimitado.

Quando usar: Para codificar funcionalidades definidas com precisão e sem inventar escopo adicional.

`implement-spec`

Execução de Spec

Implementação automática e estruturada dos artefatos gerados especificamente pelas skills /to-spec e /to-tickets.

Quando usar: Imediatamente após rodar a geração de specs e tickets para iniciar a entrega de código.

`tdd`

Test-Driven Development

Desenvolvimento guiado por testes seguindo com rigor a disciplina Red-Green-Refactor. Garante testes unitários e de integração confiáveis desde o primeiro minuto.

Quando usar: Ao criar lógicas de cálculo, transformações de dados ou algoritmos onde a corretude é primordial.

`diagnosing-bugs`

Depuração Sistemática

Metodologia estruturada para isolamento de defeitos difíceis, reprodução mínima, análise de hipóteses e eliminação definitiva de regressões e falhas de runtime.

Quando usar: Sempre que um erro inesperado, exceção silenciosa ou gargalo de lentidão for identificado.

`migrate-to-shoehorn`

Segurança de Tipos em Testes

Migra asserções de tipo inseguras ('as Type') em arquivos de teste para a biblioteca utilitária @total-typescript/shoehorn, mantendo os dados parciais com checagem segura do TypeScript.

Quando usar: Para limpar mocks frágeis em testes e elevar a confiabilidade da suíte de testes TypeScript.

`scaffold-exercises`

Criação de Exercícios

Gera estruturas completas para exercícios práticos com seções, problemas, soluções automatizadas e notas explicativas com conformidade de lint.

Quando usar: Para criar material de treino, tutoriais internos ou desafios técnicos no projeto.

🔍 5. Revisão de Código, Git & Workflows de Equipe

`code-review`

Revisão Dual

Revisa as alterações de uma branch ou commit disparando duas análises especializadas em paralelo: 'Standards' (padrões de engenharia e estilo do repo) e 'Spec' (fidelidade aos requisitos solicitados).

Quando usar: Antes de abrir um PR ou antes de mesclar alterações críticas para garantir alta qualidade.

`pr`

Documentação de PR

Gera corpos de Pull Requests profissionais, com sumário executivo das mudanças, motivações, checklist de validação e impactos.

Quando usar: Ao finalizar uma funcionalidade e preparar o Pull Request para submissão no GitHub/GitLab.

`git-guardrails-claude-code`

Segurança no Git

Configura proteções e hooks para barrar comandos destrutivos no Git (como push forçado, git reset --hard ou deleção acidental de branches).

Quando usar: Para proteger repositórios importantes contra comandos destrutivos executados por agentes autônomos ou acidentalmente.

`setup-pre-commit`

Automação Pré-Commit

Configura e padroniza hooks com Husky e lint-staged (Prettier, ESLint, checagem de tipos tsc e execução dos testes pertinentes aos arquivos alterados).

Quando usar: Para garantir que nenhum commit quebre o build ou fuja das regras de formatação do time.

`triage`

Triagem de Demandas

Conduz issues e PRs externos através de uma máquina de estados: categorização, verificação técnica, validação de bugs e elaboração de briefs prontos para execução por agentes.

Quando usar: Para organizar filas de issues, chamados de usuários ou contribuições externas de forma padronizada.

`retro`

Retrospectiva de Sessão

Facilita uma retrospectiva técnica e operacional ao final de uma sessão de programação, avaliando o que funcionou, lições aprendidas e próximos passos.

Quando usar: Ao concluir marcos relevantes ou após sessões intensas de pareamento.

6. Orquestração, Handoff & Navegação de Agentes

`ask-matt`

Roteador de Skills

Funciona como um guia inteligente sobre o repositório, indicando exatamente qual skill ou fluxo de trabalho é mais adequado para o desafio que você tem em mãos.

Quando usar: Quando estiver em dúvida sobre qual comando, skill ou abordagem adotar a seguir.

`chief-of-staff`

Coordenação de Subagentes

Atua como coordenador de longo prazo para metas complexas em uma única sessão, orquestrando subagentes especialistas e garantindo alinhamento de escopo.

Quando usar: Para tarefas multidisciplinares de longa duração que exigem múltiplos papéis.

`claude-handoff`

Transição em Background

Passa o trabalho e contexto da conversa atual para um novo agente em segundo plano, que assume a continuidade imediatamente sem perda de foco.

Quando usar: Para delegar execuções assíncronas longas ou passar o bastão para outra instância de agente.

`handoff`

Resumo de Transição

Sintetiza e compacta o estado da conversa e do código em um documento conciso de transição, pronto para ser lido e continuado por qualquer outro agente posteriormente.

Quando usar: Ao terminar seu turno de trabalho ou antes de reiniciar o contexto da IA.

`wait-what`

Recalibragem de Resposta

Interrompe a linha atual quando uma explicação ou proposta do agente não fez sentido, instruindo-o a repensar a abordagem e reexplicar sob outro ângulo.

Quando usar: Quando o assistente fornecer uma resposta confusa, prolixa ou desalinhada com a realidade do projeto.

`wizard`

Assistente Interativo

Gera scripts interativos de terminal em bash para guiar o desenvolvedor passo a passo em tarefas manuais sensíveis que apenas humanos podem fazer (configurar chaves secretas, infra, etc.).

Quando usar: Para provisionar serviços na nuvem, lidar com autenticações manuais ou migrações delicadas de banco.

`research`

Pesquisa Aprofundada

Investiga perguntas técnicas contra fontes primárias de alta confiabilidade (documentações oficiais, especificações) e registra os achados em Markdown no repositório.

Quando usar: Para pesquisar APIs desconhecidas, comparar soluções técnicas ou embasar decisões de design.

`teach`

Ensino Contextualizado

Ensina novas técnicas, conceitos de engenharia ou detalhes de bibliotecas de forma pedagógica e adaptada ao código específico do seu projeto.

Quando usar: Quando quiser entender a fundo um padrão arquitetural, biblioteca ou conceito moderno de programação.

`setup-matt-pocock-skills` Setup de Ecossistema

Prepara a infraestrutura básica do repositório para o ecossistema de engenharia: configura diretórios de documentação, templates e convenções de labels.

Quando usar: Executado na inicialização do repositório para habilitar todas as demais skills de engenharia.

✍️ 7. Escrita & Documentação Técnica

`writing-for-agents` Instruções para IAs

Diretrizes especializadas para redigir ou aprimorar arquivos de instruções consumidos por agentes de inteligência artificial (como AGENTS.md, CLAUDE.md ou novas skills).

Quando usar: Ao criar ou customizar novas skills ou regras comportamentais para o assistente.

`writing-fragments` Fase Exploratória

Fase 1 do fluxo de redação: extrai e agrupa fragmentos brutos, insights dispersos e notas sem preocupação imediata com formatação ou ordem final.

Quando usar: No início do processo de escrita técnica, reunindo ideias e referências soltas.

`writing-beats` Fase de Estruturação

Fase 2 do fluxo de redação: organiza os fragmentos em uma sequência lógica de 'batidas narrativas' (beats), assegurando que cada conceito seja definido antes de ser utilizado.

Quando usar: Para estruturar artigos, documentações de produtos ou tutoriais técnicos com progressão lógica impecável.

`writing-shape` Fase de Redação Final

Fase 3 do fluxo de redação: transforma a estrutura de batidas no artigo final completo, escrevendo parágrafo por parágrafo com coesão e clareza.

Quando usar: Para polir e finalizar artigos, manuais e guias oficiais para publicação.

8. Skills de Plataforma & Ferramentas Globais

`chrome-devtools & a11y-debugging`

DevTools & Acessibilidade

Conexão direta com o navegador via MCP: inspeção de DOM, análise de rede, depuração de erros em tempo real e auditoria de acessibilidade (WCAG, contrastes e ARIA).

Quando usar: Para depurar visualmente a aplicação e validar acessibilidade em testes com o navegador real.

`modern-web-guidance`

Web Standards

Repositório de melhores práticas modernas para HTML, CSS e JavaScript: View Transitions, Container Queries, seletores :has, otimização de renderização e APIs Web recentes.

Quando usar: Sempre que estiver construindo componentes de frontend para evitar práticas legadas ou obsoletas.

`debug-optimize-lcp & memory-leak`

Performance & Core Web Vitals

Diagnóstico de Core Web Vitals (Largest Contentful Paint) e detecção de vazamentos de memória (heap snapshots) em aplicações JavaScript.

Quando usar: Para resolver páginas lentas, melhorar métricas de carregamento ou investigar travamentos de memória.

`firebase-* (Firestore, Rules, Hosting)`

Serviços de Nuvem

Conjunto de automações para ecossistema Firebase: modelagem de coleções Firestore, autoria de regras de segurança rigorosas (firestore.rules) e deploy em Hosting.

Quando usar: Ao integrar ou manter bancos de dados em nuvem, regras de segurança e deploys estáticos.

💡 Dica de Uso com seu Assistente

Para ativar qualquer uma das habilidades acima durante a conversa, basta mencioná-la nominalmente (por exemplo: "*use a skill design-taste-frontend para melhorar esta página*" ou "*execute o code-review na minha branch*"). O assistente lerá as regras especializadas correspondentes e seguirá o fluxo rigorosamente.